



UNIVERSITY OF LEEDS

Formal Language & Finite Automata

Context-free grammars & Pushdown automata

Dr Noleen Koehler

University of Leeds

All reading is from the "Introduction to the theory of computation" -
Micheal Sipser

- Context-free languages - Pages 117 - 124 (essential)

Context-free languages

The main aim of this week is to demonstrate that CFG's are equivalent to Pushdown automata.

To do this, we will

1. Show that for every language generated by a CFG there exists a PDA that recognises it.
2. Show that for every language recognised by a PDA there exists a CFG that generates it.

Show that for every language generated by a CFG there exists a PDA that recognises it.

Context-free grammar $\implies$ Pushdown automata

Given a CFG we construct a PDA.

1. Place a marker symbol $\$$ and the start variable on the stack.
2. Repeat the following steps forever,
 - a) If the top of the stack is a variable symbol A , nondeterministically select one of the rules for A and substitute A by the string on the right-hand side of the rule.
 - b) If the top of the stack is a terminal symbol a , read the next symbol from the input and compare it to a . If they match, repeat, else reject on this branch.
 - c) If the top of the stack is the symbol $\$$, enter the accept state.

**Show that for every language
recognised by a PDA there exists
a CFG that generates it.**

Pushdown automata $\implies$ Context-free grammar

Given a pushdown automata we construct a CFG.

Firstly we require the pushdown automata to have three properties,

1. It has a single accept state q_{accept} .
2. It empties it's stack before accepting.
3. Each transition either pushes a symbol onto the stack, or pops a symbol off the stack - but not both.

How can we modify any PDA to have these three properties?

Pushdown automata $\implies$ Context-free grammar

From a PDA with these properties we then construct a grammar.

For each pair of state p, q in the PDA we introduce a variable into the grammar which will be able to generate all strings which can take the PDA from state p with an empty stack to state q with an empty stack.

Two situations can arise, either

1. The symbol popped off the stack before entering the state q is the symbol push on to the stack when exiting state p .
2. The symbol push on to the stack when exiting state p is popped at some point prior to entering state q - the stack becomes empty at an intermediate point between p and q .

We handle these cases separately.

Pushdown automata $\implies$ Context-free grammar

Let $P = (Q, \Sigma, \Gamma, \delta, q_0, \{q_{\text{accept}}\})$ that meet the conditions specified previously, we construct a grammar G such that $L(G) = L(P)$. The grammar G is constructed as follows, where $G = (V, \Sigma, R, S)$.

The variables of G are $\{A_{p,q} \mid p, q \in Q\}$. The start symbol of G is $A_{q_0, q_{\text{accept}}}$. The production rules are defined as follows:

1. For each $p, q, r, s \in Q$, $u \in \Gamma$ and $a, b \in \Sigma_\epsilon$, if $(r, u) \in \delta(p, a, \epsilon)$ and $(q, \epsilon) \in \delta(s, b, u)$ then $A_{pq} \rightarrow aA_{rs}b$ is a member of R .
2. For each $p, q, r \in Q$ we have $A_{pq} \rightarrow A_{pr}A_{rq}$ is a member of R
3. For each $p \in Q$ we have $A_{pp} \rightarrow \epsilon$ is a member of R

We must prove that $A_{pq} \xRightarrow{*} x$ if and only if x can take P from state p with an empty stack to state q with an empty stack. Each direction of the proof is considered separately.

Pushdown automata $\implies$ Context-free grammar

If A_{pq} can generate x then x can take P from state p with an empty stack to state q with an empty stack.

We prove by induction on the length of a derivation.

Pushdown automata $\implies$ Context-free grammar

If x can take P from state p with an empty stack to state q with an empty stack then A_{pq} generates x .

We prove by induction on the length of a computation.